

Threads - Exam Questions and Answers

Question 1

The price to parallelize an application whose code is 100% serial is 30 dollars per percentage unit. That is, if you invest 60 dollars, you can get the serial part of the application to 98%. At the same time, if you want to run such an application on multiple cores, every extra core, aside from the first one, costs 50 dollars.

What is better: to invest 210 dollars in parallelizing the code and 100 dollars in paying for cores, or to invest 120 dollars in parallelizing the code and 200 dollars in paying for cores? Explain why in detail.

Answer

Use Amdahl's law:

$$\text{speedup} = 1 / (S + P / N)$$

where S is the serial fraction, P is the parallel fraction, and N is the number of cores.

Option 1:

- 210 dollars spent on parallelization reduces the serial part by 7 percentage points, so $S = 0.93$ and $P = 0.07$.
- 100 dollars spent on cores buys 2 extra cores, so the application runs on 3 cores.

$$\begin{aligned}\text{speedup} &= 1 / (0.93 + 0.07 / 3) \\ &= 1 / 0.9533 \\ &= 1.049\end{aligned}$$

Option 2:

- 120 dollars spent on parallelization reduces the serial part by 4 percentage points, so $S = 0.96$ and $P = 0.04$.
- 200 dollars spent on cores buys 4 extra cores, so the application runs on 5 cores.

$$\begin{aligned}\text{speedup} &= 1 / (0.96 + 0.04 / 5) \\ &= 1 / 0.968 \\ &= 1.033\end{aligned}$$

The first option is better because it gives the larger speedup. In this case, reducing the serial part is more useful than buying more cores, because the serial fraction limits the benefit of parallel execution.

Question 2

Is the following code going to print the right value when executed with parameter 10 or not? Explain why. If you think there is something to correct, point out all the corrections that must be applied to the code in order for it to work.

The listing contains only the relevant portions of code, so your answer does not need to take minor inconsistencies, such as missing include statements, into account.

```
int sum;
void *runner(void *param);

int main(void)
{
    pthread_t tid;
    pthread_attr_t attr;

    if (argc != 2) {
        return -1;
    }

    if (atoi(argv[1]) < 0) {
        return -1;
    }

    pthread_attr_init(&attr);
    pthread_create(&tid, &attr, runner, argv[1]);

    printf("sum is %d\n", sum);
}

void *runner(void *param)
{
    int i, upper = atoi(param), sum = 0;

    for (i = 1; i <= upper; i++)
        sum += i;

    pthread_exit(0);
}
```

Answer

No, the program is not guaranteed to print the right value.

There are two main problems:

- The main thread prints `sum` immediately after `pthread_create()`. It does not wait for the created thread to finish. It should call `pthread_join(tid, NULL)` before printing.
- The `runner()` function declares a local variable named `sum`. This local variable shadows the

global `sum`, so the global variable printed by `main()` is not updated.

The relevant corrections are:

- wait for the thread with `pthread_join(tid, NULL)` before printing the result;
- update the global `sum`, or return the result to the main thread instead of storing it in a local variable;
- if considering the full compilable program, use a `main` signature that provides `argc` and `argv`, such as `int main(int argc, char *argv[])`.

Question 3

Can users use kernel threads? Motivate your answer.

Answer

Yes, user programs can use kernel threads through the threading interface provided by the operating system and its libraries. For example, a user program can create POSIX threads, and on many systems these are implemented as kernel-scheduled threads.

This does not mean that user code runs in kernel mode. It means that the threads created by the user program are represented and scheduled by the kernel. The kernel decides when these threads run, possibly on different cores.